POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY
FACULTY OF INFORMATION SECURITY

COURSE PROJECT REPORT
COURSE: FUNDAMENTALS OF INFORMATION SECURITY

Project Group: Group 9

B25DCAE104 – Giáp Đức Đạt
B25DCAE095 – Nguyễn Quang Tuấn
B25DCAE047 – Bùi Bảo Long
B25DCAE098 – Nguyễn Thành Vinh

Instructor: Dr. Nguyễn Ngọc Điệp

SEMESTER 1, ACADEMIC YEAR 2026–2027

MỤC LỤC

MỤC LỤC 2
DANH MỤC CÁC HÌNH VẼ ..... **Error! Bookmark not defined.**
& ABSTRACT 4
CHAPTER 1. INTRODUCTION & BACKGROUND 5
1.1 Context & Motivation 5
1.2 Project Scope 5
1.3 Methodology & Target Artifacts 5
CHAPTER 2. ARCHITECTURE & IMPLEMENTATION OF SECURE LOGIN APP 5
2.1 Overall Objective 5
2.2 Specific Measurable Objectives 6
CHAPTER 3. Theoretical Foundations 6
3.1 Cryptographic Hash Functions & Security Properties 6
3.2 Fast Hashes vs. Dedicated Password Hashing Functions 6
3.3 Cryptographic Salt and Pepper 6
3.4 Memory-Hard Functions 7
3.5 Rainbow Table Architecture 7
3.6 Recovery Suites: Hashcat & John the Ripper 7
CHAPTER 4. A Comparison of MD5 and Argon2id HASH 7
4.1 Why MD5 Was Abandoned? 7
4.2 Defense Mechanisms in Modern Password Hashing 8
4.3 Deep Dive: Argon2id Architecture 9
CHAPTER 5. Building a Secure Application Login (Secure App Login) 9
5.1 Architectural Design 9
5.1.1 3-Tier Software Architecture 9
5.1.2 4-Layer Password Protection Model 10
5.2 Password Storage 11
5.3 Algorithm Selection & Cost Tuning 11
5.3.1 Empirical Cost Tuning Parameters 11
5.4 Supplementary Security Measures 12
5.5 Source Code Implementation 12
5.6 Functional & Security Testing 13
CHAPTER 6. Implementation and Attack Demonstration 14

6.1 Experimental Overview 14
6.2 Tool and Environment Configuration 14
6.3 Methodology and Workflow 14
6.4 Results and Observations 15
CHAPTER 7. Results & Empirical Analysis 15
7.1 Quantitative Performance Synthesis 15
7.2 Root Cause Analysis 15
7.3 Performance vs. Security Trade-Offs 15
7.4 Experimental Limitations 16

& ABSTRACT

Insecure password storage remains one of the primary causes behind large-scale data breaches. Historically, numerous systems relied on fast cryptographic hash functions such as MD5 or SHA-1. Because these algorithms were engineered for data integrity verification rather than credential storage, attackers can recover original plaintext passwords within seconds to minutes upon acquiring an offline database dump.

This report presents a controlled experimental study conducted entirely within an isolated lab environment to:

1. Quantitatively compare six widely used hashing algorithms — MD5, SHA-1, PBKDF2, bcrypt, scrypt, and Argon2id — across cost factor parameters, memory consumption, and computation execution time.
2. Build a reference authentication application complying with modern security standards (OWASP ASVS / NIST SP 800-63B), incorporating cryptographically secure pseudo-random number generator (CSPRNG) 128-bit salts, an isolated application pepper (HMAC-SHA256), and cost tuning for Argon2id ($m = 64$ MiB, $t = 3, p = 2$).
3. Execute a controlled password cracking demonstration on a purely synthetic dataset devoid of real personally identifiable information (PII) to illustrate the stark disparity in adversarial computational cost.
4. Provide concrete architectural recommendations, migration strategies (lazy rehashing), and professional ethical commitments adhering to Vietnamese legal frameworks.

CHAPTER 1. INTRODUCTION & BACKGROUND

1.1 Context & Motivation

In modern identity and access management (IAM), password authentication remains ubiquitous yet represents the most targeted attack surface. Storing passwords in plaintext constitutes an egregious vulnerability. However, merely hashing passwords with fast cryptographic algorithms (e.g., MD5, SHA-1, standard SHA-256) offers negligible protection against modern adversaries equipped with high-throughput hardware (GPUs, ASICs, FPGAs) capable of evaluating billions of candidate hashes per second.

1.2 Project Scope

All experimental procedures within this study are strictly bounded within an isolated sandbox environment: virtual machines or containerized workloads running on a dedicated private VLAN, completely disconnected from live production networks, public infrastructure, or unauthorized third-party services. The password dataset leveraged for the cracking demonstration is entirely synthetic, procedurally generated via automated scripts, and contains no real-world compromised data or personal identifiers.

1.3 Methodology & Target Artifacts

The subject of this study encompasses six cryptographic hashing algorithms, a secure authentication application, and two industry-standard password recovery utilities (Hashcat and John the Ripper). The methodology couples theoretical cryptographic design analysis with quantitative empirical benchmarking on fixed hardware specifications. Complete reproducibility is guaranteed via version-controlled scripts, configuration files, and automated deployment recipes hosted on a private GitHub repository.

CHAPTER 2. ARCHITECTURE & IMPLEMENTATION OF SECURE LOGIN APP

2.1 Overall Objective

To achieve an in-depth operational and cryptographic understanding of password hashing functions, design and deploy an authentication microservice adhering to modern defense-in-depth principles, and visually demonstrate—via controlled offline cracking—why algorithm selection and parameter tuning govern real-world credential resilience.

2.2 Specific Measurable Objectives

No.	Objective	Measurement Criteria
1	A Comparison of MD5 and Argon2id Hash Algorithms	Why Was MD5 Abandoned, Defense Mechanisms in Modern Password Hashing
2	Deploy a secure authentication application featuring per-user salt, server pepper, and cost tuning	Fully functional web service with automated test suites verifying positive/negative authentication
3	Execute offline cracking attacks on MD5/SHA-1 using Hashcat/John the Ripper on a synthetic dataset	Measured attack throughput (H/s) and elapsed recovery time
4	Benchmark cracking resistance of Argon2id against the same dataset under fixed time constraints	Measured throughput drop and projected time-to-crack estimations

CHAPTER 3. THEORETICAL FOUNDATIONS

3.1 Cryptographic Hash Functions & Security Properties

An ideal cryptographic hash function mapping arbitrary input $\{0,1\}^* \rightarrow \{0,1\}^n$ must satisfy three fundamental properties:

1. **Preimage Resistance (One-wayness):** Given a hash digest h, it is computationally infeasible to find any message m such that $\text{Hash}(m) = h$.
2. **Second-Preimage Resistance (Weak Collision Resistance):** Given an input m_1, it is computationally infeasible to discover an alternative input $m_2 \neq m_1$ such that $\text{Hash}(m_1) = \text{Hash}(m_2)$.
3. **Collision Resistance (Strong Collision Resistance):** It is computationally infeasible to discover any arbitrary pair of distinct inputs (m_1, m_2) such that $\text{Hash}(m_1) = \text{Hash}(m_2)$.

3.2 Fast Hashes vs. Dedicated Password Hashing Functions

General-purpose hashing algorithms (MD5, SHA-1, SHA-256) minimize CPU overhead to support high-throughput data pipelines and digital signatures. Conversely, specialized password derivation functions (Key Derivation Functions – KDFs) such as PBKDF2, bcrypt, scrypt, and Argon2id are deliberately engineered to be computationally demanding and memory-intensive, thereby drastically escalating the hardware expense and power consumption required per candidate trial.

3.3 Cryptographic Salt and Pepper

Salt: A cryptographically secure, unique pseudo-random byte string generated per user, appended to the plaintext password prior to hashing, and stored openly alongside the digest

in the database. Salts invalidate precomputed dictionary structures (rainbow tables) by forcing attackers to build dedicated tables for each unique salt value.

Pepper: An application-wide secret symmetric key applied to credentials prior to hashing. Unlike salts, peppers are strictly stored outside the database layer (e.g., dedicated key management services, environment variables, or Hardware Security Modules). In the event of an offline database compromise, the absence of the pepper prevents attackers from initiating dictionary attacks.

3.4 Memory-Hard Functions

Algorithms like scrypt and Argon2id mandate extensive, continuous RAM allocation per hashing operation. While standard CPU/GPU cores excel at massively parallel, compute-bound tasks, scaling memory-hard algorithms encounters memory bandwidth bottlenecks and VRAM capacity constraints on GPU/ASIC architectures, neutralizing hardware advantages historically leveraged by adversaries.

3.5 Rainbow Table Architecture

A rainbow table utilizes precalculated chains of hash reductions to trade off storage for computation time, facilitating rapid plaintext recovery of unsalted hashes. Salt injection renders this strategy economically non-viable, as each distinct salt demands an entirely distinct multi-gigabyte rainbow table generation.

3.6 Recovery Suites: Hashcat & John the Ripper

Hashcat delivers kernel-level parallel compute optimization across modern GPUs, supporting hundreds of hash formats (-m) alongside attack modes (dictionary, combinator, mask, and hybrid rule-based attacks). John the Ripper provides extensible CPU-optimized rule manipulation engines. Both utilities serve as authorized security evaluation and penetration-testing tools when deployed strictly on owned, sandboxed systems.

CHAPTER 4. A COMPARISON OF MD5 AND ARGON2ID HASH

4.1 Why MD5 Was Abandoned?

Algorithmic Vulnerabilities & Collision Resistance Breakdown

MD5 (Message Digest Algorithm 5), designed by Ronald Rivest in 1991, processes 512-bit message blocks in a Merkle–Damgård construction to produce a 128-bit digest. Its decline stems from both architectural and mathematical weaknesses:

- **Analytical Collisions:** In 2004, Xiaoyun Wang et al. demonstrated practical collision attacks against MD5, breaking its collision resistance ($H(m_1) = H(m_2)$ with $m_1 \neq m_2$). Today, generating an MD5 collision takes mere seconds on commodity hardware.
- **Length Extension & Truncation Flaws:** The Merkle–Damgård construction is fundamentally vulnerable to length-extension attacks when improperly used in naive Message Authentication Code (MAC) schemes without HMAC wrapping.

Extreme Execution Speed (Throughput as a Flaw)

While high throughput is desirable when checking the integrity of a 10 GB ISO file, it is disastrous for low-entropy inputs like user passwords:

- An off-the-shelf modern consumer GPU (e.g., NVIDIA RTX 4090) computes **tens of billions to hundreds of billions of MD5 hashes per second** using tools like Hashcat.
- If a database of raw or even salted MD5 hashes is exfiltrated, an attacker can exhaust typical password character spaces in minutes.

Vulnerability to Lookup Structures

- **Dictionary & Brute-Force Attacks:** Standard 8-character alphanumeric passwords have an entropy of roughly ≈ 47.5 bits. At a speed of 100 GH/s, exhaustive search finishes in less than two seconds.
- **Precomputed Rainbow Tables:** Because MD5 has a fixed internal state with zero native cost-tuning or memory requirements, unsalted hashes allow precomputed space-time trade-off matrices (rainbow tables) to invert hashes instantaneously.

4.2 Defense Mechanisms in Modern Password Hashing

Modern password hashing algorithms invert the speed priority, operating on the principle of **deliberate slowness and memory exhaustion**.

Work Factor & Resource Asymmetry

To protect low-entropy inputs, a hashing algorithm must force the verifier (and thus the attacker) to spend significant physical resources per attempt. This is governed by three primary dimensions of cost:

1. **Time Cost (t):** The number of iterative passes executed over the internal state.
2. **Memory Cost (m):** The volume of RAM allocated and continuously accessed during evaluation.
3. **Parallelism Cost (p):** The number of concurrent threads/lanes required to process the calculation.

Salting & Pepper Strategies

- **Cryptographic Salt:** A unique, cryptographically secure pseudo-random value (minimum 16 bytes) generated per credential and stored alongside the hash.
    - **Role:** Defeats precomputed rainbow tables and ensures identical passwords yield entirely distinct hashes across accounts, forcing attackers to target hashes strictly one-by-one.
- **Pepper (Secret Keying):** A high-entropy secret key stored external to the database (e.g., in an HSM, KMS, or isolated environment variable) and mixed into the hash via HMAC or pre-hashing ($K \oplus$ input).

- **Role:** If a database is dumped via SQL injection, the attacker cannot begin offline cracking until they also breach the separate environment hosting the pepper.

Memory-Hard Functions (MHF) & Cache-Timing Protection

Older slow hashing schemes (PBKDF2, pure bcrypt) are primarily CPU-bound. Attackers circumvent them by deploying custom ASICs or FPGAs with tiny arithmetic units running in parallel.

- **Memory-Hardness:** Modern algorithms allocate tens or hundreds of megabytes of RAM per hash. Because memory chips (SRAM/DRAM) require silicon surface area and high memory bandwidth, an attacker cannot pack thousands of cracking cores onto an ASIC without running into physical memory bandwidth and die-size limits.
- **Side-Channel Defense:** Data-dependent memory access patterns (like those in scrypt or Argon2d) leak information through CPU cache hit/miss timings. True modern hashers provide data-independent access or hybrid modes to neutralize cache-timing side channels.

4.3 Deep Dive: Argon2id Architecture

Argon2, selected as the winner of the Password Hashing Competition (PHC) in July 2015 and standardized under **RFC 9106**, provides three main variants:

- **Argon2d:** Uses data-dependent memory access. Highly resistant to GPU cracking, but vulnerable to side-channel cache-timing attacks (best suited for backend cryptographic currencies or proof-of-work where side channels are irrelevant).
- **Argon2i:** Uses data-independent memory access. Fully immune to cache-timing attacks, but marginally less resistant to tradeoff attacks on memory density.
- **Argon2id (Hybrid Standard):** The universally recommended standard for user authentication. It operates as **Argon2i during the first half of the first pass** over memory to neutralize side-channel leaks, and then transitions to **Argon2d for the remainder of the execution** to maximize resistance against GPU/ASIC Time-Memory Trade-Off (TMTO) attacks.

CHAPTER 5. BUILDING A SECURE APPLICATION LOGIN (SECURE APP LOGIN)

5.1 Architectural Design

The registration and login system is designed following a 3-Tier Software Architecture combined with a 4-Layer Defense-in-Depth Model, ensuring modularity, seamless cloud deployment (Render Cloud), and stringent attack surface management.

5.1.1 3-Tier Software Architecture

- **Presentation Tier:** A single-page interface built with HTML5, CSS3, and JavaScript, communicating with the backend server via JSON. This tier handles credential collection during login/registration, displays client-side validation warnings, and manages the Lockout Countdown timer upon receiving rate-limiting responses from the server.
- **Business Logic & Security Tier:** Built on **Python Flask** and served via **Gunicorn WSGI Server** (--workers 1 --threads 4). This layer includes the ProxyFix middleware to resolve real client IP addresses behind the Load Balancer, Sliding Window Rate Limiting controls, HIBP compromised-password filtering, a Pepper secret pre-processing module (HMAC-SHA256), and the Argon2id hashing engine.
- **Dynamic Data Storage Tier:** Manages the synthetic user dataset (compliant with CLO3) via an in-memory dynamic array structure (DYNAMIC_DATASET) with automatic persistent synchronization to a users_dataset.json file.

5.1.2 4-Layer Password Protection Model

- **Transport & Network Identification Layer:** Enforces end-to-end encryption for all authentication traffic via HTTPS/TLS and extracts the originating client IP address using the X-Forwarded-For header (Werkzeug ProxyFix).
- **Online Defense Layer:** Defends against Brute Force attacks via a Sliding Window Rate Limiting mechanism, thwarts Credential Stuffing using k-Anonymity queries, and eliminates Timing Side-Channel leaks using a dummy hashing technique (DUMMY_HASH).
- **Application Secret Key Layer:** Blends raw user passwords with a server-side secret key (PEPPER_SECRET) using HMAC-SHA256 prior to feeding into the primary hashing algorithm.
- **Hardware-Resistant Hashing & Storage Layer:** Hashes the pre-processed output using the **Argon2id** algorithm with a cryptographically unique 16-byte Salt per user before persisting records to disk.

5.2 Password Storage

The system strictly adheres to **OWASP ASVS (Section V2.4 - Credential Storage)** and **NIST SP 800-63B** guidelines. Storing passwords in plaintext or utilizing fast cryptographic hash functions (such as MD5, SHA-1, or unsalted SHA-256) is strictly prohibited.

The password transformation pipeline prior to storage follows a two-stage cryptographic construction:

PreHash = HMAC-SHA256(key = PEPPER_SECRET, message = Password_UTF-8)StoredHash = Argon2id(input = PreHash, salt = CSPRNG(16 bytes), m = 65536, t = 3, p = 2)

5.3 Algorithm Selection & Cost Tuning

Among current password hashing standards (PBKDF2-HMAC-SHA256, bcrypt, scrypt, Argon2), Argon2id (version v=19, RFC 9106) was chosen as the core hashing algorithm based on several technical factors:

- Hybrid Execution: Argon2id blends the strengths of Argon2d (data-dependent memory access for maximum GPU/ASIC/FPGA resistance) during the initial pass and Argon2i (data-independent memory access to prevent cache-timing side-channel attacks) during subsequent passes.
- Overcoming Legacy Weaknesses:

  Compared to PBKDF2: PBKDF2 relies almost exclusively on CPU cycles (CPU-hard) with minimal memory footprint (< 4 KB), allowing adversaries to leverage GPU clusters to evaluate tens of thousands of hashes per second.

  Compared to bcrypt: bcrypt enforces a rigid 72-byte input length limitation (risking truncation issues with long passphrases or pre-processed inputs) and uses a static 4 KB RAM footprint.

5.3.1 Empirical Cost Tuning Parameters

Cost tuning was benchmarked so that single login authentications on the server execute within **~100 ms – 250 ms** (imperceptible latency for legitimate users while drastically degrading offline brute-force throughput):

Configuration Parameter	Symbol	Configured Value	Configuration Parameter
Memory Cost	m	65536 KiB (64 MiB)	Memory Cost
Time Cost / Iterations	t	3	Time Cost / Iterations
Parallelism	p	2	Parallelism
Salt Length	s	16 bytes (128-bit)	Salt Length
Output / Hash Length	L	32 bytes (256-bit)	Output / Hash Length

5.4 Supplementary Security Measures

Beyond offline storage hardening, the application incorporates 4 proactive defense mechanisms against online attack vectors:

- **Sliding Window Rate Limiting (5 attempts / 3 minutes):**
  - Failed authentication attempts are tracked across two dimensions: Client IP address (ip:<client_ip>) and Target Username (user:<username>).
  - If an IP or user account registers 5 or more failed login attempts within 180 seconds, subsequent requests are short-circuited at the controller layer with an HTTP 429 Too Many Requests status and remaining cooldown time (retry_after), bypassing Argon2id computation to protect server CPU/RAM from DoS attacks.
  - Upon successful login, failed counters for that session are reset (clear_failed_attempts).
- **Compromised Password Verification via k-Anonymity:**
  - On the registration endpoint (/api/register), the user's password is submitted to SHA-1 hashing (40 uppercase hexadecimal characters).
  - The application splits the hash into a 5-character prefix to query the compromised index (HIBP_INDEX) and evaluates the remaining 35-character suffix locally. Mirroring the *1Password / Have I Been Pwned* architecture, this rejects weak and leaked passwords without ever transmitting raw credentials over the network.
- **Mitigation of Timing Side-Channels & User Enumeration:**
  - If a requested username does not exist, an immediate error response would take only ~0.5 ms, whereas an existing user (with an incorrect password) incurs the full Argon2id execution of ~150 ms. Attackers can exploit this 149.5 ms timing delta to enumerate valid accounts.
  - To counter this, non-existent accounts trigger a dummy verification step: ph.verify(DUMMY_HASH, apply_pepper(password)) utilizing identical cost parameters (m=64MiB, t=3, p=2), returning a uniform error response: *"Invalid username or password"*.

5.5 Source Code Implementation

The complete registration/login source code is located in the src/app/ directory within the project's GitHub repository. It contains the aforementioned Argon2id configuration, REST endpoints (/api/register, /api/login, /api/dataset), and rate-limiting middleware.

5.6 Functional & Security Testing

TC-01	Valid Registration & Salt Uniqueness	Register two accounts (user_a and user_b) with identical password MatKhauManh@2026.	Returns HTTP 200. users_dataset.json contains two distinct argon2id... records with unique Salt and Hash values.	Pass
TC-02	Block Compromised Passwords (HIBP k-Anonymity)	Register an account using known leaked passwords (123456, qwerty123, admin123).	Registration is rejected with HTTP 422 Unprocessable Entity and an explicit breach warning.	Pass
TC-03	Positive Login Verification	Authenticate as admin with valid credentials AnToan@2026.	HMAC-SHA256 + Argon2id verification succeeds (HTTP 200), failure counters clear, redirected to dashboard.	Pass
TC-04	Brute Force Mitigation via Rate Limit (5 / 3 min)	Input 5 consecutive incorrect passwords for admin within 30 seconds.	Attempts 1–4 return HTTP 401 with remaining retry counters (4 down to 1). Attempt 5 triggers HTTP 429, locking the login button with a 180s countdown.	Pass
TC-05	Account Enumeration & Timing Attack Defense	Measure API /api/login response latency between a non-existent account (ghost_user) and an existing account (admin) on invalid passwords.	Both requests yield an identical HTTP 401 error message and comparable processing times (~140 ms – 180 ms, variance $\Delta t < 10$ ms) due to DUMMY_HASH.	Pass
TC-06	Offline Leak Resistance (Hashcat)	Extract argon2id$v=19$m=65536,t=3,p=2... hashes from users_dataset.json and execute dictionary cracking via hashcat.	Attack is infeasible within the lab testing window due to the 64 MiB RAM footprint per hash and absence of the server-side PEPPER_SECRET.	Pass

CHAPTER 6. IMPLEMENTATION AND ATTACK DEMONSTRATION

6.1 Experimental Overview

To illustrate the vulnerability of unsalted MD5 hashing against precomputation attacks, we conducted a proof-of-concept demonstration using a rainbow table methodology. The scenario simulates an adversary obtaining a leaked database containing unsalted MD5 credential hashes and recovering the original plaintexts using precomputed lookup structures.

6.2 Tool and Environment Configuration

- Precomputation Utility: `knotty.php` was utilized to process raw dictionary wordlists into structured rainbow tables (hash-to-plaintext chains).
- Wordlist Source: A standard plaintext wordlist file containing commonly used passwords.
- Target Algorithm: MD5 (Message Digest Algorithm 5), operating without salt parameters.
- Simulated Leak Dataset: A mock repository containing compromised user records with columns `[user_id, md5_hash]`.

6.3 Methodology and Workflow

Rainbow Table Generation:

- The input plaintext dictionary (wordlist.txt) was fed into knotty.php.
- The script computed cryptographic reduction functions and hash chains across multiple iterations, generating compact table files mapping target hash domains back to plaintext inputs.
- By trading disk storage and offline precomputation time against online lookup speed, the resulting tables enable near-instantaneous reversal of common hashes.

Ingestion of Compromised Credentials:

- A simulated target dataset containing leaked MD5 hashes was ingested into the processing pipeline.
- Duplicate entries were filtered, and target hashes were indexed for sequential lookup.

Hash Inversion and Cracking:

- The recovery module executed reduction and lookup cycles against the precomputed table generated by knotty.php.

- For hashes matching known trajectories within the table chains, false-alarm reduction checks were applied to confirm valid pre-images, successfully restoring the original plaintexts.

6.4 Results and Observations

The precomputation approach demonstrated that unsalted MD5 digests offer negligible defense against offline lookup techniques. Common strings and dictionary entries were inverted within seconds, confirming that legacy hash functions lacking adaptive work factors and salt values fail to protect user credentials once database exfiltration occurs.

CHAPTER 7. RESULTS & EMPIRICAL ANALYSIS

7.1 Quantitative Performance Synthesis

Benchmarking demonstrates an insurmountable computational chasm between legacy hashing algorithms and memory-hard KDFs:

1. MD5 / SHA-1: Hashcat achieves computation rates spanning tens of billions of hashes per second ($\boldsymbol{GH/s}$). Unsalted synthetic hashes across dictionary spaces were cracked virtually instantaneously ($\boldsymbol{< 1}$ second).
2. bcrypt (cost $= \mathbf{12}$): Throughput drops precipitously to tens of thousands of hashes per second ($\boldsymbol{kH/s}$), elevating dictionary attacks from seconds to multiple days.
3. Argon2id ($\boldsymbol{m = 64}$ MiB, $\boldsymbol{t = 3, p = 2}$): Throughput collapses to $<$ $\mathbf{100}$ hashes/second per GPU compute unit. Hardware memory saturation and memory-bus bottlenecks severely throttle parallel scaling.

7.2 Root Cause Analysis

The divergence in recovery resilience stems from three structural design choices:

1. Memory-Hardness: Argon2id and scrypt force heavy RAM allocation per candidate check, starving GPU streaming multiprocessors of local memory and nullifying massive SIMD parallelism.
2. Algorithmic Cost Scaling: Increasing work factors forces attackers to execute CPU-intensive iterative calculations per candidate trial.
3. Cryptographic Salt & Pepper Integration: Per-user salts render rainbow tables useless, while pepper isolation forces attackers to compromise server memory before initiating offline cracking workflows.

7.3 Performance vs. Security Trade-Offs

System design requires balancing cryptographic friction against acceptable user experience. While higher work factors harden against brute-force attacks, excessive verification latencies (> 500 ms) degrade web responsiveness and increase vulnerability to authentication-based Denial-of-Service (DoS) attacks. Setting Argon2id parameters to

maintain server execution times between $150\ \text{ms}$ and $250\ \text{ms}$ establishes a strong equilibrium between user experience and attack resistance.

7.4 Experimental Limitations

- **Dataset Magnitude:** The synthetic dataset ($N = 1{,}000$ accounts) reflects a smaller footprint than enterprise databases holding millions of user records, slightly altering lookup cache dynamics.

- **Hardware Boundary:** Experiments executed on a single GPU/CPU node do not reflect the computational capacity of distributed botnets or ASIC clusters. Absolute crack durations serve as comparative pedagogical metrics rather than absolute guarantees of security.